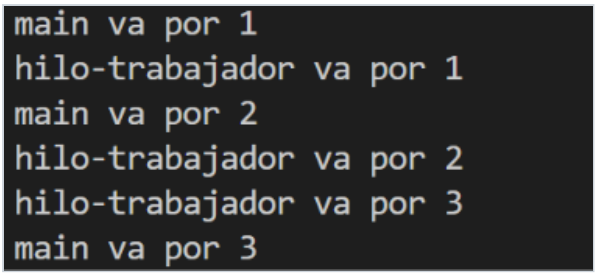
# Cuaderno de actividades

PSP-UD2 · RA2

## Actividad 1 · Del proceso al hilo: crear, esperar y parar

### Actividad 1.1 · start() frente a run()

**CAPTURA · SALIDAS DE LA TERMINAL CON START() Y CON RUN()**




--- con start() ---




--- con run() ---

**4 · DIFERENCIA ENTRE START() Y RUN() CITANDO LAS DOS SALIDAS**

**Con `start()` hay dos hilos trabajando a la vez.**

- La salida sale entrelazada: `main va por 1`, `hilo-trabajador va por 1`, `main va por 2`...
- `start()` pone en marcha el hilo-trabajador, que ejecuta la tarea mientras main ejecuta la suya con `tarea.run()`.
- Por eso unas líneas dicen hilo-trabajador y otras main: cada hilo que ejecuta la tarea imprime su propio nombre.

**Con `run()` solo trabaja main.**

- Todas las líneas dicen main y salen seguidas, sin mezclarse: `main va por 1, 2, 3`, dos veces.
- `run()` no crea ningún hilo. Es una llamada a un método normal, y la ejecuta quien la escribe: main.
- El hilo-trabajador se ha creado, pero nunca se ha arrancado. Por eso `join()` vuelve al momento y el mensaje `main: el trabajador ha terminado` no es cierto.

---

**Conclusión:** `start()` crea un hilo nuevo que ejecuta la tarea en paralelo; `run()` solo ejecuta la tarea en el hilo que ya está corriendo.

### Actividad 1.1 · Pasos 5 y 6 · tres hilos sobre la misma tarea

**CAPTURA · LA SALIDA: LA ÚLTIMA LÍNEA DEL MAIN SALE LA ÚLTIMA**

```
hilo1 va por 1
hilo3 va por 1
hilo2 va por 1
hilo3 va por 2
hilo1 va por 2
hilo2 va por 2
hilo3 va por 3
hilo2 va por 3
hilo1 va por 3
main: las 3 tareas han terminado.
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2>
```

## Actividad 1.3 · El reparto de la UD1, ahora entre hilos

**9 · MEDIDAS CON N = 1, 2, 4 Y 8 (CALENTAMIENTO DESCARTADO, CINCO PASADAS, MEDIANA)**

Portátil de 4 núcleos, con el equipo en reposo.

```
TOTAL 1 hilos: 25938 ms total=1270607
TOTAL 1 hilos: 25973 ms total=1270607
TOTAL 1 hilos: 25899 ms total=1270607
TOTAL 1 hilos: 25980 ms total=1270607
TOTAL 1 hilos: 25797 ms total=1270607
TOTAL 2 hilos: 16598 ms total=1268420
TOTAL 2 hilos: 16424 ms total=1268435
TOTAL 2 hilos: 16516 ms total=1268366
TOTAL 2 hilos: 16369 ms total=1268376
TOTAL 2 hilos: 16400 ms total=1268381
TOTAL 4 hilos: 8927 ms total=1264155
TOTAL 4 hilos: 8927 ms total=1264183
TOTAL 4 hilos: 8838 ms total=1264142
TOTAL 4 hilos: 8880 ms total=1264150
TOTAL 4 hilos: 8938 ms total=1264170
TOTAL 8 hilos: 7199 ms total=1263523
TOTAL 8 hilos: 7171 ms total=1263459
```

```
TOTAL 8 hilos: 7199 ms total=1263523
TOTAL 8 hilos: 7171 ms total=1263459
TOTAL 8 hilos: 7171 ms total=1263291
TOTAL 8 hilos: 7174 ms total=1263302
TOTAL 8 hilos: 7726 ms total=1263625
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a1-hilos>
```

| N hilos | Pasadas (ms) | Mediana (ms) | total en la mediana | Primos que faltan |
|---|---|---|---|---|
| 1 | 25938 · 25973 · 25899 · 25980 · 25797 | 25938 | 1270607 | 0 |
| 2 | 16598 · 16424 · 16516 · 16369 · 16400 | 16424 | 1268435 | 2172 |
| 4 | 8927 · 8927 · 8838 · 8880 · 8938 | 8927 | 1264155 | 6452 |
| 8 | 7199 · 7171 · 7171 · 7174 · 7726 | 7174 | 1263302 | 7305 |

**10 · MEDIDAS DE HILOS AL LADO DE LAS DE PROCESOS DE LA UD1 (MISMO RANGO)**

| N | Procesos UD1 · mediana (ms) | Hilos · mediana (ms) | Hilos · total de primos |
|---|---|---|---|
| 1 | 28010 | 25938 | 1270607 |
| 2 | 17473 | 16424 | 1268435 |
| 4 | 10698 | 8927 | 1264155 |
| 8 | 10585 | 7174 | 1263302 |

**11 · EQUIPO Y CONDICIONES DE LA MEDIDA**

- **Equipo:** portátil HP Envy x360, Windows 11 Home, 7,39 GB de RAM, 4 núcleos.
- **Java:** 21.0.12 LTS (el mismo que en las medidas de procesos de la UD1).
- **Rango:** [2, 20.000.000), el mismo de la UD1.
- **Ejecución:** desde la terminal de VS Code con `java PrimosHilos.java N`, para N = 1, 2, 4 y 8.
- **Método:** una pasada de calentamiento descartada, cinco pasadas por cada N y me quedo con la mediana.
- **Condiciones:** equipo en reposo, con el navegador y el resto de programas cerrados.

**12 · ¿CUÁL ARRANCA ANTES, EL REPARTO ENTRE PROCESOS O ENTRE HILOS? ¿CUÁL ESCALA MEJOR?**

Arranca antes el reparto entre hilos. Crear un hilo es barato porque vive dentro del mismo proceso y comparte su memoria; un proceso tiene que arrancar una JVM nueva. Se ve con N=1: 25.938 ms con hilos frente a 28.010 ms con procesos.

Escalan mejor los hilos: con 4 mejoran 2,91x (procesos 2,62x) y con 8 mejoran 3,62x (procesos 2,65x, prácticamente igual que con 4, porque 8 JVM compiten por 4 núcleos y la memoria). Pero los hilos dan un total incorrecto (con 8 faltan 7.305 primos), así que de momento son más rápidos dando un resultado que está mal.

**13 · ¿POR QUÉ EL TROZO 0 TERMINA SIEMPRE MUCHO ANTES QUE EL ÚLTIMO?**

Porque los trozos tienen la misma cantidad de números, pero no el mismo trabajo. `esPrimo(x)` prueba divisores hasta la raíz de x: con los números pequeños del trozo 0 termina enseguida, y con los números grandes del último trozo (cerca de 20.000.000) tiene que probar muchos más. Repartir el rango en partes iguales no reparte la carga por igual: el trozo 0 acaba pronto y su núcleo se queda sin nada que hacer, mientras el último sigue trabajando, y es ese trozo el que marca el tiempo total.

## Actividad 1.4 · El servicio del club y su limpiador

**16 · CÓDIGO: LAS LÍNEAS DEL MAIN QUE PARAN EL LIMPIADOR Y LO ESPERAN**

```
Thread.sleep(args.length > 0 ? Long.parseLong(args[0]) : 3000);
//Actividad 1.4 Paso 16
limpiador.interrupt(); //tras dejar correrlo 3s lo interrumpimos
limpiador.join(); //lo esperamos.
```

**16 · CAPTURA: LA TRAZA DEL ARRANQUE Y LA PARADA DEL SERVICIO**

```
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a1-hilos> java ServicioClub.java
13:17:46.185 [main] servicio del club arrancado
13:17:46.242 [limpiador] vuelta de limpieza: empiezo
13:17:47.120 [limpiador] franja 5 liberada (era de bruno)
13:17:47.532 [limpiador] vuelta de limpieza: termino sin dejar nada a medias
13:17:48.535 [limpiador] vuelta de limpieza: empiezo
13:17:48.753 [limpiador] franja 2 liberada (era de ana)
13:17:49.306 [limpiador] vuelta de limpieza: termino sin dejar nada a medias
13:17:49.307 [limpiador] limpiador detenido
13:17:49.307 [main] main termina
```

## Actividad 1.5 · Qué lleva hilo y qué no

**17 · CLASIFICACIÓN DE LAS SEIS OPERACIONES DEL SERVICIO**

| Operación del club | A qué espera | Grupo | ¿Lleva hilo? ¿Por qué? |
|---|---|---|---|
| Comprobar si una franja está libre | A nada: es leer una posición de memoria | Instantánea | **No.** No espera a nada; el hilo costaría más que la operación. |
| Liberar las franjas caducadas cada segundo | Al reloj | Espera mucho | **Sí.** Espera al reloj; duerme en segundo plano sin bloquear el servicio. |
| Enviar el correo de confirmación de una reserva | A un servidor de correo | Espera mucho | **Sí.** Espera al servidor de correo; así el usuario no espera con él. |
| Calcular cuántos primos hay en el rango común | A nada: es CPU pura, pero se trocea | Calcula mucho | **Sí.** CPU pura: se reparte entre hilos, uno por núcleo. |
| Formatear la fecha de una reserva para la pantalla | A nada: microsegundos | Instantánea | **No.** Microsegundos; no espera a nada. |
| Escribir el registro del día en un fichero | Al disco | Espera mucho | **Sí.** Espera al disco; el servicio sigue mientras se escribe. |

18 · OPERACIONES QUE NO LLEVAN HILO: QUÉ SE GANARÍA Y POR QUÉ NO COMPENSA

**Comprobar si una franja está libre**

- **Qué se ganaría:** nada. Es leer una posición de memoria y acaba al instante.
- **Por qué no compensa:** crear y arrancar un hilo cuesta mucho más que la propia lectura, y habría que esperarlo con `join()` para saber la respuesta.
- **Además:** otro hilo más tocando `franja[]` agrava la carrera de la sección crítica.

**Formatear la fecha de una reserva para la pantalla**

- **Qué se ganaría:** nada. Tarda microsegundos y no espera a nada.
- **Por qué no compensa:** el hilo tardaría más en crearse que la operación en hacerse.
- **Además:** la pantalla necesita el resultado ya, así que habría que esperarlo igual.

# Actividad 2 · La carrera, demostrada con un número

## Actividad 2.1 · Las predicciones

1 · (A) DOS HILOS INCREMENTAN UN CONTADOR UN MILLÓN DE VECES CADA UNO. ¿QUÉ SALE?

El contador incrementado 2 millones de veces: 2.000.000.

2 · (B) EN MIL INTENTOS DE RESERVAR LA MISMA FRANJA CON DOS HILOS, ¿CUÁNTAS VECES SE CONFIRMARÁN LAS DOS?

Cada vez: las 1.000 veces.

## Actividad 2.2 · El contador

4 · CONTADOR.JAVA EJECUTADO VARIAS VECES SEGUIDAS

```
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1572517  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1373856  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1742444  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1378968  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1807670  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1843890  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera>
```